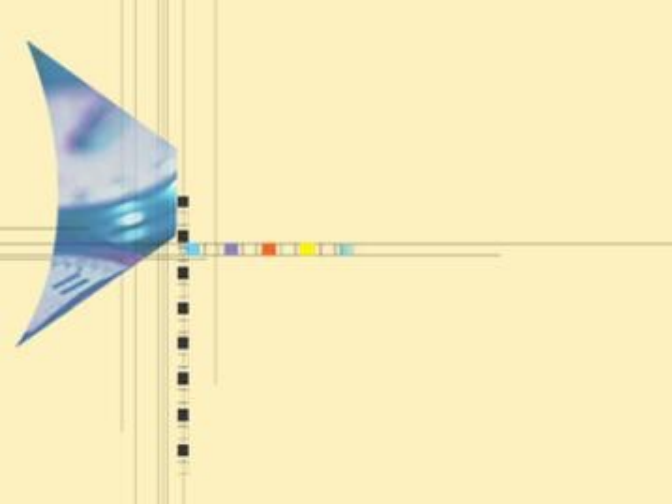




算法设计与分析

主讲：刘娟，武汉大学人工智能学院

2024-2025 第二学期，1-16周

周一(3-4，三区1-502)，周三（6-7，三区1-325）

2025/4/1

武汉大学计算机学院

武汉大学
Wuhan University





主要内容

5.1 引言

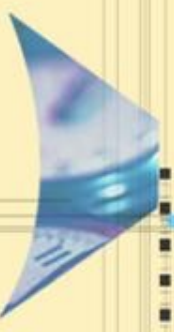
5.2 贪心算法简单例

5.3 贪心法设计思想

5.4 贪心法正确性证明

5.5 对贪心法得不到最优解情况的处理

5.6 贪心策略的典型应用



主要内容

5.3 贪心算法设计思想

5.3.1 贪心算法基本思想

5.3.2 贪心准则重要性

5.3.1 贪心算法基本思想

- 贪心算法在少量计算的基础上做出正确猜想而不急于考虑以后的情况。
- 它一步步地来构筑解。每一步均是建立在局部最优解的基础之上，而每一步又都扩大了部分解的规模。做出的选择产生最大的直接收益而又保持可行性。
- 因为每一步的工作很少且基于少量信息，所得算法特别有效。

贪心算法一般框架

Algorithm GREEDY(A,n)

 solution $\leftarrow \emptyset$

 for $i \leftarrow 1$ to n do

$x \leftarrow \text{SELECT}(A)$

 if FEASIBLE(solution,x)

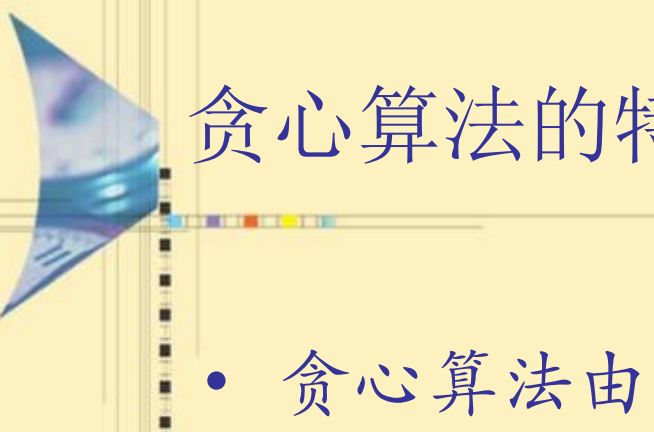
 then solution $\leftarrow \text{UNION}(\text{solution},x)$

 end if

 end for

 return (solution)





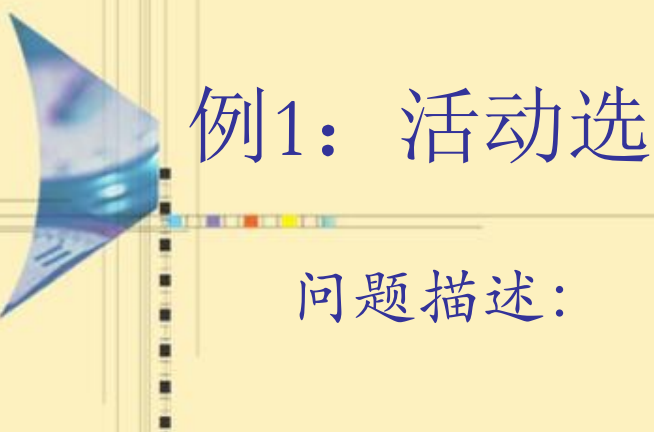
贪心算法的特性

- 贪心算法由一个简单的迭代过程构成;
- 在维持可行性的前提下它选择能产生最大直接利益的项。
- 贪心算法产生全局最优解的最重要因素是选择策略, 即采用的贪心准则/贪心度量。

5.3.2 贪心准则的重要性

- 例1: 活动选择问题
- 例2: 最小延迟问题
- 例3: 分数背包问题





例1：活动选择问题

问题描述：

有 n 项活动申请使用同一个礼堂，每项活动有个开始时间和一个截止时间。任何两个活动不能同时举行。

问：如何选择这些活动，从而使被安排的活动数量达到最多？

活动选择问题建模

$S = \{1, 2, \dots, n\}$ 为 n 项活动的集合,

s_i, f_i 分别为活动 i 的开始和结束时间

定义: 活动 i 与 j 相容 当且仅当 $s_i \geq f_j$ 或 $s_j \geq f_i$

求最大的两两相容的活动集 A

策略1: 排序使得 $s_1 \leq s_2 \leq \dots \leq s_n$, 从前向后挑选

策略2: 排序使得 $f_1 - s_1 \leq f_2 - s_2 \leq \dots \leq f_n - s_n$, 从前向后挑选

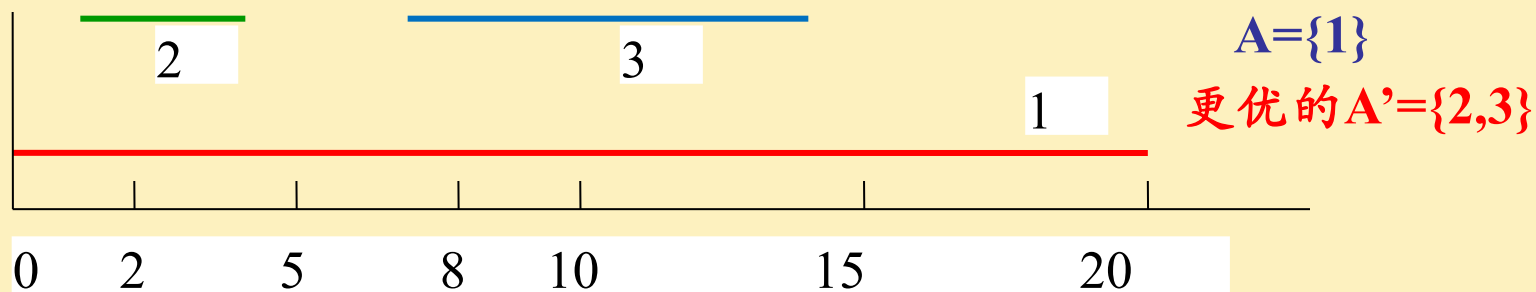
策略3: 排序使得 $f_1 \leq f_2 \leq \dots \leq f_n$, 从前向后挑选

以上策略中的挑选都要注意满足相容性条件。

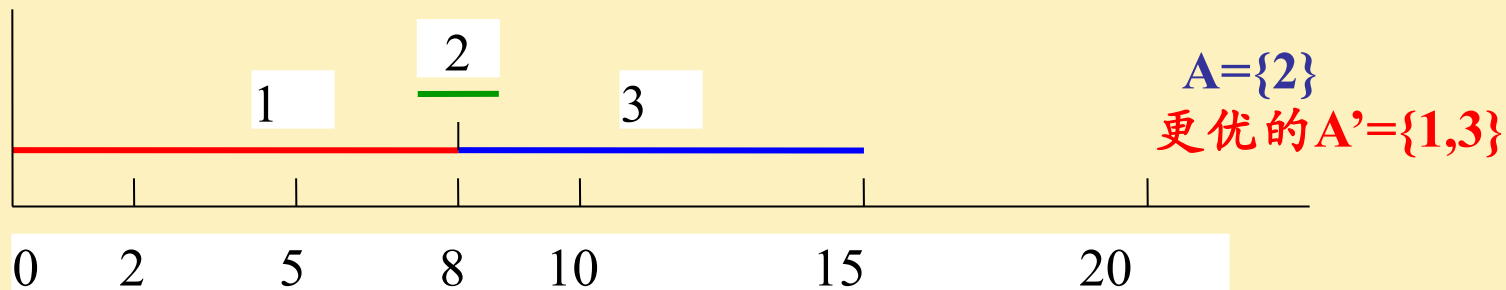
求解的过程就是一个多步判断的过程, 策略就是规定了选择的依据。

策略1和策略2的两个反例

策略1: $S=\{1,2,3\}$, $s_1=0$, $f_1=20$, $s_2=2$, $f_2=5$, $s_3=8$, $f_3=15$



策略2: $S=\{1,2,3\}$, $s_1=0$, $f_1=8$, $s_2=7$, $f_2=9$, $s_3=8$, $f_3=15$



贪心算法: 按截止时间排序

算法5.6 Greedy-Select

1. $n \leftarrow \text{length}[S];$
2. $A \leftarrow \{1\};$
3. $j \leftarrow 1;$
4. for $i \leftarrow 2$ to n
5. if $s_i \geq f_j$ then //相容性判断
6. $A \leftarrow A \cup \{i\};$
7. $j \leftarrow i;$
8. end if
9. end for
10. return $A.$

算法时间复杂度 $O(n)$

最后完成时间 $t = \max \{ f_k : k \in A \}.$

实例

输入

i	1	2	3	4	5	6	7	8	9	10	11
s_i	1	3	0	5	3	5	6	8	8	2	12
f_i	4	5	6	7	8	9	10	11	12	13	14

解为 $A = \{1, 4, 8, 11\}$ $t=14$

例2：最小延迟调度问题

- 特定等待服务的客户集 $S=\{1, 2, \dots, n\}$ ，预计对客户 i 的服务时间为 t_i ，该客户希望的完成时间为 d_i 。假定 t_i ， d_i 都是正整数。
- 如果对客户 i 的服务在 d_i 之前完成，那么对该客户的服务没有延迟，如果在 d_i 之后结束，那么这个服务就被延迟了，延迟的时间等于服务结束的时间减去 d_i 。
- 求对客户最大延迟时间最小的服务调度顺序

例2：最小延迟调度问题

问题建模：

任务集合 S , $\forall i \in S$, d_i 为截止时间, t_i 为加工时间, d_i, t_i 为正整数.

一个调度函数 $f: S \rightarrow N$, $f(i)$ 为任务 i 的开始时间.

求最大延迟达到最小的调度, 即求 f 使得:

$$\min_f \{ \max_{i \in S} \{ f(i) + t_i - d_i \} \}$$

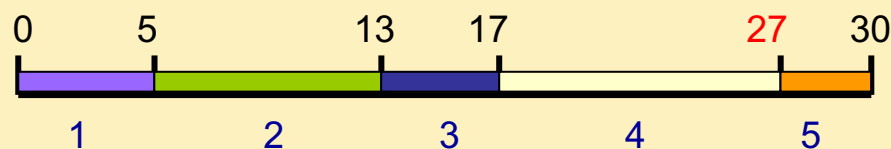
$$\forall i, j \in S, i \neq j, f(i) + t_i \leq f(j) \text{ or } f(j) + t_j \leq f(i)$$

实例

$S=\{1, 2, 3, 4, 5\}$, $d=<10, 12, 15, 11, 20>$, $t=<5, 8, 4, 10, 3>$

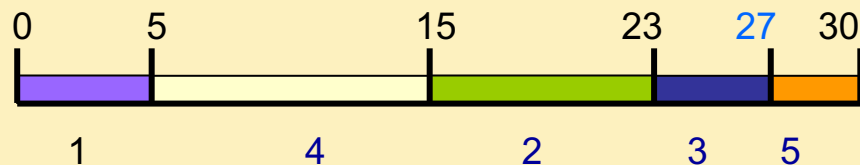
调度1: $f_1(1)=0$, $f_1(2)=5$, $f_1(3)=13$, $f_1(4)=17$, $f_1(5)=27$

各任务延迟: 0, 1, 2, 16, 10; 最大延迟: 16



调度2: $f_2(1)=0$, $f_2(2)=15$, $f_2(3)=23$, $f_2(4)=5$, $f_2(5)=27$

各任务延迟: 0, 11, 12, 4, 10; 最大延迟: 12



贪心策略选择

贪心策略1: 按照 t_i 从小到大安排任务

贪心策略2: 按照 $d_i - t_i$ 从小到大安排任务

贪心策略3: 按照 d_i 从小到大安排任务

策略1 对某些实例得不到最优解.

反例: $t_1=1, d_1=100, t_2=10, d_2=10$, $\{1,2\}$ 延迟1, 不是最优

策略2 对某些实例得不到最优解.

反例: $t_1=1, d_1=2, t_2=10, d_2=10$ $\{2,1\}$ 延迟10, 不是最优

算法设计

算法思想:

按照截止时间 d_i 从小到大选择任务, 安排时不留空闲时间

算法 5.7 JOB

1. Sort(S) 使得 $d_1 \leq d_2 \leq \dots \leq d_n$
2. $f(1) \leftarrow 0$, $i \leftarrow 2$
3. while $i \leq n$ do
4. $f(i) \leftarrow f(i-1) + t_{i-1}$
5. $i \leftarrow i+1$

时间复杂度为排序的时间复杂度

例3：分数背包问题

问题描述

- 给出 n 个大小为 $s_1, s_2, \dots, s_n$ ，值为 $v_1, v_2, \dots, v_n$ 的物品，并设背包的容量为 C 。要求找到非负实数 $x_1, x_2, \dots, x_n$ ，使得在满足不超过背包总容量的约束条件下，装进背包物品部分总价值最大。

- 即，最大化 $\sum_{i=1}^n x_i v_i$ 且满足 $\sum_{i=1}^n x_i s_i \leq C$

实例

- $n=3$, $C=20$, $(v_1, v_2, v_3)=(25, 24, 15)$,
 $(s_1, s_2, s_3)=(18, 15, 10)$

- 可行解:

(x_1, x_2, x_3)	$\sum s_i x_i$	$\sum v_i x_i$
(1)(1/2, 1/3, 1/4)	16.5	24.5
(2)(1, 2/15, 0)	20	28.2
(3)(0, 2/3, 1)	20	31
(4)(0, 1, 1/2)	20	31.5

选择策略-1

- 贪心准则：从剩下的物品中选择最大价值的物品装入背包。
- 得到可行解 (2)(1,2/15,0)20 28.2 (非最优)
- 原因：虽然价值很大，但是容量的消耗太快

选择策略-2

- 贪心准则：从剩下的物品中选择最小尺寸的物品装入背包。
- 得到可行解(3) $(0, 2/3, 1)$ 20 31 (非最优)
- 原因：虽然容量损耗较少，但是价值增长速度太慢

选择策略-3

- 贪心准则：从剩下的物品中选择最大 v_i/s_i 比率的商品装入背包（单位大小的价值最大的物品）。
- 得到可行解 (4) (0,1,1/2) 20 31.5 (最优)

最终确定的分数背包问题算法

- Step 1:
 - 对每个物品，计算 $y_i = v_i/s_i$ ，即计算物品的价值和大小的比。
- Step 2:
 - 再按比值的降序来排序所有物品，从第一个物品开始装背包，然后是第二个物品，依次类推。如果能全部放就全部放进去，不能全部放就尽可能地多放，直至装满背包。

算法5.8 Greedy-KNAPSACK

输入: 按照 $v(i)/s(i)$ 比率降序排列的 n 个物品的大小数组 $s[1..n]$ 和物品价值数组 $v[1..n]$; 背包的总容量 C , 物品的数量 n

输出: 最优的装包结果 $x[1..n]$

```
for  $i \leftarrow 0$  to  $n$     {初始化 $x[i]$  }  
     $x[i] \leftarrow 0$   
end for  
 $cu \leftarrow C$   
for  $i \leftarrow 0$  to  $n$     {物品按次序装入背包 }  
    if  $s[i] > cu$  then  
        exit  
    end if  
     $x(i) \leftarrow 1$   
     $cu \leftarrow cu - s(i)$  {剩余背包容量}  
end for  
if  $i \leq n$  then  $x(i) \leftarrow cu/s(i)$  end if {最后剩余背包容量中装入物品 $i$ 的部分}  
return  $x$ 
```




主要内容

5.1 引言

5.2 贪心算法简单例

5.3 贪心法设计思想

5.4 贪心法正确性证明

5.5 对贪心法得不到最优解情况的处理

5.6 贪心策略的典型应用

贪心算法的正确性证明

(1) 归纳法：证明贪心法得到最优解

叙述一个描述正确性的命题

对算法步数归纳或者对问题规模归纳

(2) 交换论证法：在保证最优性不变的前提下，从一个最优解进行逐步替换，最终得到贪心法的解

例1: 活动选择问题-贪心算法正确性证明

活动选择问题问题描述:

有 n 项活动申请使用同一个礼堂, 每项活动有个开始时间和一个截止时间。任何两个活动不能同时举行。

问: 如何选择这些活动, 从而使被安排的活动数量达到最多?

贪心算法: 按截止时间升序排序

算法5.6 Greedy-Select

1. $n \leftarrow \text{length}[S];$
2. $A \leftarrow \{1\};$
3. $j \leftarrow 1;$
4. for $i \leftarrow 2$ to n
5. if $s_i \geq f_j$ then //相容性判断
6. $A \leftarrow A \cup \{i\};$
7. $j \leftarrow i;$
8. end if
9. end for
10. return $A.$

算法Greedy-Select正确性证明

定理 算法Greedy-Select 执行到第 k 步, 选择 k 项活动 $i_1=1, i_2, \dots, i_k$, 那么存在最优解 A 包含 $i_1=1, i_2, \dots, i_k$

根据定理: 算法至多到第 n 步得到最优解

归纳证明（对算法步骤进行归纳）

归纳基础：证明存在最优解包含活动 1

归纳步骤：假设按照算法前 k 步选择都导致最优解，证明第 $k+1$ 步选择也导致最优解

归纳步骤的思路：

1. 算法第 k 步选择活动 $i_1=1, i_2, \dots, i_k$ ，根据归纳假设，存在最优解： $A=\{i_1=1, i_2, \dots, i_k\} \cup B$
 B 是剩下的待选活动集 S' 的一个最优解
2. 由归纳基础，存在 S' 的最优解 B' 包含 i_{k+1}
3. 由 $|B'|=|B|$ 知 $A'=\{i_1=1, i_2, \dots, i_k\} \cup B'$ 最优
4. $A'=\{i_1=1, i_2, \dots, i_k, i_{k+1}\} \cup (B'-\{i_{k+1}\})$ 最优.

证明：归纳基础

设 $S=\{1,2,\dots,n\}$ 是活动集，活动按截止时间递增顺序排序。

$k=1$, 证明存在最优解包含活动1.

任取最优解 A , A 中的活动按照截止时间递增的顺序排列。

如果 A 的第一个活动为 j , $j \neq 1$, 令

$$A' = (A - \{j\}) \cup \{1\},$$

由于 $f_1 \leq f_j$, 因此, 活动1和 $i_2, \dots, i_k$ 等活动都相容, A' 和 A 中活动个数相同, 于是 A' 也是最优解, 且含有1.

证明：归纳步骤

假设命题对 k 为真,证明对 $k+1$ 也为真.

算法执行到第 k 步, 选择了活动 $i_1=1, i_2, \dots, i_k$, 根据归纳假设存在最优解 A 包含 $i_1=1, i_2, \dots, i_k$,

A 中剩下的活动选自集合 $S'=\{i \mid i \in S, s_i \geq f_k\}$, 且

$$A = \{i_1, i_2, \dots, i_k\} \cup B$$

B 是 S' 的最优解. (若不然, S' 的最优解为 B^* , B^* 的活动比 B 多, 那么 $B^* \cup \{1, i_2, \dots, i_k\}$ 是 S 的最优解, 且比 A 的活动多, 与 A 的最优性矛盾.)

根据归纳基础, 存在 S' 的最优解 B' 含有 S' 中的第一个活动, 设为 i_{k+1} , 且 $|B'|=|B|$, 于是

$$\{i_1, i_2, \dots, i_k\} \cup B' = \{i_1, i_2, \dots, i_k, i_{k+1}\} \cup (B' - \{i_{k+1}\})$$

也是原问题的最优解.

例2：最优装载问题-贪心算法正确性证明

问题描述： n 个集装箱 $1, 2, \dots, n$ 装上轮船，集装箱 i 的重量 w_i ，轮船装载重量限制为 c ，无体积限制。问如何装使得上船的集装箱最多？不妨设 $w_i \leq c$ 。

$$\max \sum_{i=1}^n x_i$$

$$\sum_{i=1}^n w_i x_i \leq c$$

$$x_i = 0, 1 \quad i = 1, 2, \dots, n$$

贪心策略：将集装箱按照从轻到重排序，轻者先装

贪心选择性质证明（对问题规模归纳）

命题：对任何规模为 n (n 是正整数) 的输入，上述贪心法都得到最优解.

证明思路 对问题规模归纳

- 设集装箱标号按照从轻到重记为 $1, 2, \dots, n$
- $n=1$, 贪心选择得到最优解（只有1个箱子，不证）
- 假设对于规模为 $n-1$ 的输入得到最优解，证明对规模为 n 的输入也得到最优解

归纳步骤

假设对于 $n-1$ 个集装箱的输入，贪心法都可以得到最优解，考虑 n 个集装箱的输入 $N = \{1, 2, \dots, n\}$, 其中

$$w_1 \leq w_2 \leq \dots \leq w_n.$$

由归纳假设，对于 $N' = \{2, 3, \dots, n\}$, $c' = c - w_1$, 贪心法 得到最优解 I' .
令 $I = \{1\} \cup I'$, 则 I 是关于 N 的最优解.

若不然，存在包含 1 的关于 N 的最优解 I^* （如果 I^* 中没有 1，用 1 替换 I^* 中的第一个元素得到的解也是最优解），且 $|I^*| > |I|$ ；
那么 $I^* - \{1\}$ 是 N' 的解且

$$|I^* - \{1\}| > |I - \{1\}| = |I'|$$

与 I' 的最优性矛盾.

说明

- 用归纳法证明贪心算法正确性时，需要叙述一个相关命题
- 如果对算法步数归纳，命题的主要内容：对于任何正整数 k ，贪心算法的前 k 步都导致最优解
- 如果对问题规模归纳，命题的主要内容：对于任何正整数 k ，贪心算法对于规模为 k 的实例都得到最优解。
- 除了归纳法，也可以使用交换论证的方法来证明。

例3：最小延迟调度问题-贪心算法正确性证明

问题描述：

- 特定等待服务的客户集 $S=\{1, 2, \dots, n\}$ ，预计对客户 i 的服务时间为 t_i ，该客户希望的完成时间为 d_i 。假定 t_i ， d_i 都是正整数。
- 如果对客户 i 的服务在 d_i 之前完成，那么对该客户的服务没有延迟，如果在 d_i 之后结束，那么这个服务就被延迟了，延迟的时间等于服务结束的时间减去 d_i 。
- 求对客户最大延迟时间最小的服务调度顺序

最小延迟调度问题

问题建模:

任务集合 S , $\forall i \in S$, d_i 为截止时间, t_i 为加工时间, d_i, t_i 为正整数.

一个调度函数 $f: S \rightarrow N$, $f(i)$ 为任务 i 的开始时间.

求最大延迟达到最小的调度, 即求 f 使得:

$$\min_f \{ \max_{i \in S} \{ f(i) + t_i - d_i \} \}$$

$$\forall i, j \in S, i \neq j, f(i) + t_i \leq f(j) \text{ or } f(j) + t_j \leq f(i)$$

算法设计

算法思想：

按照截止时间 d_i 从小到大选择任务，安排时不留空闲时间

算法5.7 JOB

1. Sort(S) 使得 $d_1 \leq d_2 \leq \dots \leq d_n$
2. $f(1) \leftarrow 0$, $i \leftarrow 2$
3. while $i \leq n$ do
4. $f(i) \leftarrow f(i-1) + t_{i-1}$
5. $i \leftarrow i+1$

正确性证明（交换论证）

算法的解的性质： 没有空闲时间，没有逆序.

- 逆序 (i, j) : $f(i) < f(j)$ and $d_i > d_j$

引理 所有没有逆序、没有空闲时间的调度具有相同的最大延迟.

证: 因为 f_1 与 f_2 都没有逆序, 具有相同截止时间的任务必须被连续安排. 在这些连续安排的任务中最大延迟是最后一个任务, 被延迟的时间只与已安排任务加工时间之和有关, 与任务标号无关.

设相同截止时间的客户 $i_1, i_2, \dots, i_k$ 从 t_0 开始连续安排, 截止时间均为 d , 则最大延迟的客户是最后一个客户 i_k , 延迟时间为:

$$t_0 + (t_{i_1} + t_{i_2} + \dots + t_{i_k}) - d$$

这个值与 $i_1, i_2, \dots, i_k$ 的排列顺序无关

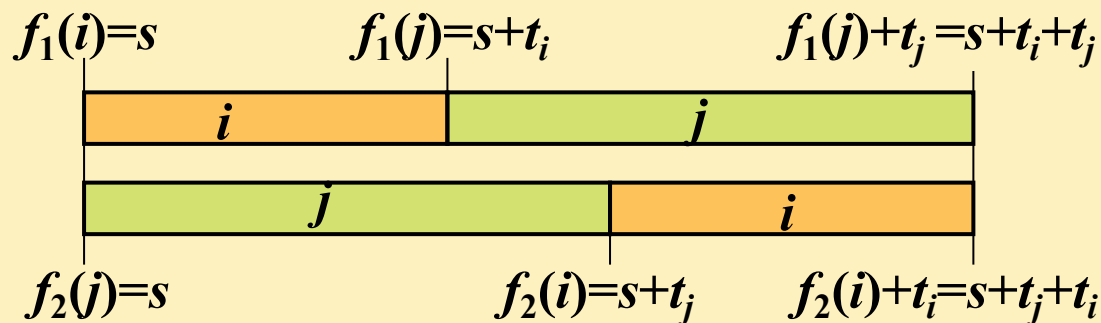
交换论证的证明方法

证明思想：从一个没有空闲时间的最优解出发，在不改变最优性的条件下，转变成没有逆序的解。

- (1) 如果一个最优调度存在逆序，那么存在 $i < n$ 使得 $(i, i+1)$ 构成一个逆序。
- (2) 存在逆序 (i, j) , $j=i+1$ ，那么交换 i 和 j 得到的解的逆序数减1，**后面证明这个新的调度仍旧最优。**
- (3) 至多经过 $n(n-1)/2$ 次交换得到一个没有逆序的最优调度。（任何排列的逆序总数不超过 $n(n-1)/2$ ）

交换相邻逆序任务*(i, j)*不影响最优性

- (1) 交换 *i, j* 对其他任务的延迟时间没影响
- (2) 交换后不增加 *j* 的延迟
- (3) 任务 *i* 在 f_2 的延迟 L_{2i} 小于任务 *j* 在 f_1 的延迟 L_{1j} , 因此小于 f_1 的最大延迟



i 在 f_2 结束时间 = *j* 在 f_1 的结束时间 = $s+t_i+t_j$

i 在 f_1 的延迟: $L_{1i} = (s+t_i)-d_i$

i 在 f_2 的延迟: $L_{2i} = (s+t_i+t_j)-d_i$

j 在 f_1 的延迟: $L_{1j} = (s+t_i+t_j)-d_j$

j 在 f_2 的延迟: $L_{2j} = (s+t_j)-d_j$

$$d_j < d_i \Rightarrow L_{2i} < L_{1j}$$

例4: 分数背包问题贪心算法正确性证明

问题描述

- 给出 n 个大小为 $s_1, s_2, \dots, s_n$, 值为 $v_1, v_2, \dots, v_n$ 的物品, 并设背包的容量为 C 。要求找到非负实数 $x_1, x_2, \dots, x_n$, 使得在满足不超过背包总容量的约束条件下, 装进背包物品部分总价值最大。

- 即, 最大化 $\sum_{i=1}^n x_i v_i$ 且满足 $\sum_{i=1}^n x_i s_i \leq C$

分数背包问题-贪心策略

- Step 1:
 - 对每个物品，计算 $y_i = v_i/s_i$ ，即计算物品的价值和大小的比。
- Step 2:
 - 再按比值的降序来排序所有物品，从第一个物品开始装背包，然后是第二个物品，依次类推。如果能全部放就全部放进去，不能全部放就尽可能地多放，直至装满背包。

算法5.8 Greedy-KNAPSACK

输入: 按照 $v(i)/s(i)$ 比率降序排列的 n 个物品的大小数组 $s[1..n]$ 和物品价值数组 $v[1..n]$; 背包的总容量 C , 物品的数量 n

输出: 最优的装包结果 $x[1..n]$

```
for  $i \leftarrow 0$  to  $n$     {初始化 $x[i]$  }  
     $x[i] \leftarrow 0$   
end for  
 $cu \leftarrow C$   
for  $i \leftarrow 0$  to  $n$     {物品按次序装入背包 }  
    if  $s[i] > cu$  then  
        exit  
    end if  
     $x(i) \leftarrow 1$   
     $cu \leftarrow cu - s(i)$  {剩余背包容量}  
end for  
if  $i \leq n$  then  $x(i) \leftarrow cu/s(i)$  end if {最后剩余背包容量中装入物品 $i$ 的部分}  
return  $x$ 
```

正确性证明（交换论证）

定理： 如果 $v_1/s_1 \geq v_2/s_2 \geq \dots \geq v_n/s_n$, 则算法Greedy-KNAPSACK对于给定的分数背包问题实例，生成一个最优解。

证明：

- 设 $X=(x_1, x_2, \dots, x_n)$ 是Greedy-KNAPSACK算法所生成的解。如果所有的 x_i 等于1，显然这个解就是最优解。
- 假设 j 是使得 $x_j \neq 1$ 的最小下标。由算法可知，对于 $1 \leq i < j$, 均有 $x_i=1$ ；对于 $j < i \leq n$, 均有 $x_i=0$ ；对于 j , 有 $0 \leq x_j < 1$ 。
- 假设有一个最优解为 $Y=(y_1, y_2, \dots, y_n)$, 则必有 $\sum y_i v_i \geq \sum x_i v_i$ 。不失一般性，可以假定 $\sum y_i s_i = M$ 。
- 如果 $Y \neq X$, 则它们必然存在不相等的元素。假设第一个不相等的元素下标为 k , 即 k 是使得 $x_k \neq y_k$ 的最小的 k 。
- 如下可证: $y_k < x_k$

正确性证明（交换论证）

- (1) 若 $k < j$, 则必有 $x_k = 1$ 。又由于 $x_k \neq y_k$, 因此有 $y_k < x_k$
- (2) 若 $k = j$, 由于 $\sum x_i s_i = M$ 且对于 $1 \leq i < j$ 有 $x_i = y_i = 1$, 而对 $j < i \leq n$ 有 $x_i = 0$ 。如果 $y_k > x_k$, 显然有 $\sum y_i s_i > M$, 与 Y 是可行解矛盾。又由于使得 $x_k \neq y_k$, 则必然有 $y_k < x_k$
- (3) 若 $k > j$, 则有 $\sum y_i s_i > M$, 这也与 Y 是可行解矛盾, 不可能。

- 将 Y 中的 y_k 替换为 x_k , 导致总大小增加了 $(x_k - y_k)s_k$, 因此必须从 $(y_{k+1}, \dots, y_n)$ 中减去同样的大小, 使得总大小仍然为 M 。这样导致一个新的可行解 $Z = (z_1, z_2, \dots, z_n)$, 其中, $z_i = x_i$ ($1 \leq i \leq k$), 并且 $\sum_{(k < i \leq n)} (y_i - z_i)s_i = (z_k - y_k)s_k$ 。因此, 对于 Z 有
$$\begin{aligned}\sum_{1 \leq i \leq n} v_i z_i &= \sum_{1 \leq i \leq n} v_i y_i + (z_k - y_k)s_k * v_k/s_k - \sum_{k < i \leq n} (y_i - z_i)s_i * v_i/s_i \\ &\geq \sum_{1 \leq i \leq n} v_i y_i + (z_k - y_k)s_k * v_k/s_k - \sum_{k < i \leq n} (y_i - z_i)s_i * v_k/s_k \\ &= \sum_{1 \leq i \leq n} v_i y_i\end{aligned}$$

正确性证明（交换论证）

- 由于Y是最优解，因此Z也是最优解。
- Z与X更相近。如果 $Z=X$ ，则说明X也是最优解；
- 否则重复上述过程，直到将Y转换成X。由于每次转化都保持解的最优性不变。因此，X也是最优解。



主要内容

5.1 引言

5.2 贪心法设计思想

5.3 贪心法正确性证明

5.4 对贪心法得不到最优解情况的处理

5.5 贪心策略的典型应用



得不到最优解的处理方法

(1) 讨论对于哪些输入贪心选择能够得到最优解

--输入应该满足的条件

(2) 讨论贪心法的解最坏情况下与最优解的误差

--绝对误差与相对误差估计



例：找零钱问题

问题描述：

设有 n 种零钱，重量分别为： $w_1, w_2, \dots, w_n$ ，价值分别为： $v_1=1, v_2, \dots, v_n$ ($v_1 < v_2 < \dots < v_n$).

需要用这些硬币付款的总钱数是： Y

问：如何付钱使得所付钱的总重最轻？

假设币值 and 钱数都是正整数，设选用第 i 种硬币的数目为 x_i ,

$$\min\{w_1x_1 + w_2x_2 + \dots + w_nx_n\}$$

$$v_1x_1 + v_2x_2 + \dots + v_nx_n = Y$$

$$x_i \in \mathbf{N}, \quad i = 1, 2, \dots, n$$

动态规划算法

属于整数规划问题，动态规划算法可以得到最优解。

设 $F_k(y)$ 表示用前 k 种零钱，总钱数为 y 的最小重量，则

递推方程

$$F_{k+1}(y) = \min_{0 \leq x_{k+1} \leq \left\lfloor \frac{y}{v_{k+1}} \right\rfloor} \{F_k(y - v_{k+1}x_{k+1}) + w_{k+1}x_{k+1}\}$$

$$F_1(y) = w_1 \left\lfloor \frac{y}{v_1} \right\rfloor = w_1 y \quad y = 0, 1, \dots, Y$$

Greedy算法

贪心准则：优选单位价值重量最小的硬币

假设 $\frac{w_1}{v_1} \geq \frac{w_2}{v_2} \geq \dots \geq \frac{w_n}{v_n}$ ，则贪心法选择时，尽量使用标号大的硬币。

设允许使用前 k 种零钱，总钱数为 y 时，贪心法得到的总重为 $G_k(y)$ 。则有如下递推公式：

$$G_{k+1}(y) = w_{k+1} \left\lfloor \frac{y}{v_{k+1}} \right\rfloor + G_k(y \bmod v_{k+1}) \quad k=1,2,\dots,n-1; y=0,1,\dots,Y$$

$$G_1(y) = w_1 \left\lfloor \frac{y}{v_1} \right\rfloor = w_1 y \quad y=0,1,\dots,Y$$

在可选硬币标号 $1,2,\dots,k+1$ 的硬币中，尽可能用第 $k+1$ 种硬币，最多个数为 $\lfloor y/v_{k+1} \rfloor$

$k=1, 2$ 时得到最优解, $F_k(y) = G_k(y)$

$k=1$ 只有一种零钱, $F_1(y) = G_1(y) = w_1 y$

$k=2$, 使用价值大的钱越多, 得到的解越好

最优解的递推式:
$$F_2(y) = \min_{0 \leq x_2 \leq \lfloor y/v_2 \rfloor} \{F_1(y - v_2 x_2) + w_2 x_2\}$$

$$\begin{aligned} & [F_1(y - v_2(x_2 + \delta)) + w_2(x_2 + \delta)] \\ & - [F_1(y - v_2 x_2) + w_2 x_2] \\ &= [w_1(y - v_2 x_2 - v_2 \delta) + w_2 x_2 + w_2 \delta] \\ & - [w_1(y - v_2 x_2) + w_2 x_2] \\ &= -w_1 v_2 \delta + w_2 \delta = \delta(-w_1 v_2 + w_2) \leq 0 \end{aligned}$$

表明递推式中值随 x_2 增加单调减少。

x_2 取最大值时(贪心方法确定的 x_2), 达到最优(小)值。

$$w_1/v_1 \geq w_2/v_2 \Rightarrow w_1 v_2 \geq w_2 v_1 = w_2$$

$k > 2$ 时得到最优解的判定条件

定理 假定 $G_k(y) = F_k(y)$,

$$v_{k+1} > v_k \text{ 且 } v_{k+1} = pv_k - \delta, \quad 0 \leq \delta < v_k, \quad p \in \mathbb{Z}^+,$$

则以下命题等价.

$$(1) G_{k+1}(y) \leq G_k(y)$$

$$(2) G_{k+1}(y) = F_{k+1}(y)$$

$$(3) G_{k+1}(pv_k) = F_{k+1}(pv_k)$$

$$(4) w_{k+1} + G_k(\delta) \leq pw_k$$

条件(4)可用来判断对于 $k+1$ 种硬币, 贪心法能否得到最优解的充分必要条件, 且该条件与 y 的具体值无关, 实际验证可行。

用条件(4), 需 $O(k)$ 时间验证 $G_{k+1}(y) = F_{k+1}(y)$ (主要是计算 $G_k(\delta)$ 时间) 对 n 种零钱作出验证, 可在 $O(n^2)$ 时间内完成

实例

$$v_{k+1} = pv_k - \delta, \quad 0 \leq \delta < v_k, \quad p \in \mathbb{Z}^+$$
$$w_{k+1} + G_k(\delta) \leq pw_k$$

例 $v_1=1, v_2=5, v_3=14, v_4=18, w_i=1, i=1, 2, 3, 4.$

对一切 y 有 $G_1(y)=F_1(y), G_2(y)=F_2(y).$

验证 $G_3(y) = F_3(y)?$

$$v_3 = pv_2 - \delta \Rightarrow 14 = 5p - \delta \Rightarrow p=3, \delta=1.$$

$$w_3 + G_2(\delta) = 1 + G_2(1) = 1 + 1 = 2$$

$$pw_2 = 3 \times 1 = 3$$

$$w_3 + G_2(\delta) \leq p w_2$$

$G_3(y)$ 是最优解

实例

$$v_{k+1} = pv_k - \delta, \quad 0 \leq \delta < v_k, \quad p \in \mathbb{Z}^+$$
$$w_{k+1} + G_k(\delta) \leq pw_k$$

例 $v_1=1, v_2=5, v_3=14, v_4=18, w_i=1, i=1, 2, 3, 4.$

验证 $G_4(y) = F_4(y)?$?

$$v_4 = pv_3 - \delta \Rightarrow 18 = 14p - \delta \Rightarrow p=2, \delta=10$$

$$w_4 + G_3(\delta) = 1 + G_3(10) = 1 + 2 = 3$$

$$pw_3 = 2 \times 1 = 2$$

$w_4 + G_3(\delta) > pw_3$, $G_4(y)$ 不是最优解.

$G_4(pv_3) > F_4(pv_3)$. 即

$$G_4(28) = \lfloor 28/18 \rfloor + \lfloor 10/5 \rfloor = 1 + 2 = 3$$

$$F_4(28) = 28/14 = 2.$$